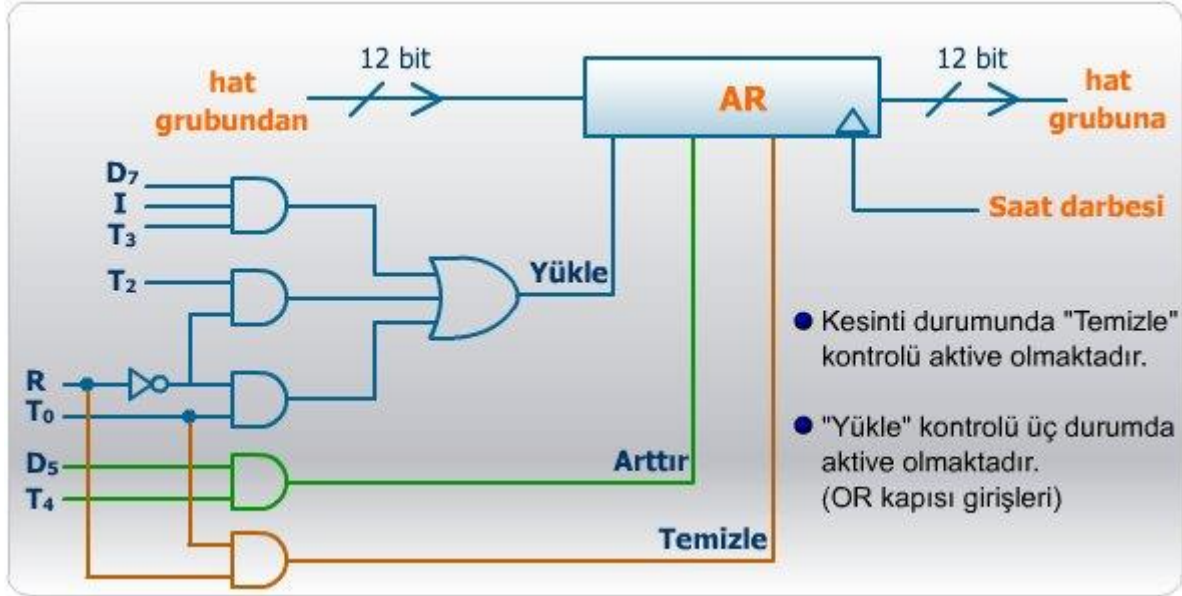


$$\text{Arttır (AR)} = D_5T_4$$

Aşağıda AR belleğinin kontrol kapıları gösterilmiştir. Tasarım K-map indirgemesinden sonra minimum sayıda eleman ile yapılır.



AR belleğinin kontrol kapıları

Diğer tüm bilgisayar bileşenleri için benzer yoldan kontrol birimi tasarımı gerçekleştirilir.

Sonuçta, oluşan yapı, hızlı bellek transferlerini belirtilen zamanlama ve koşullarda gerçekleyerek, mikroişlemleri ve dolayısıyla temel bilgisayar için tanımlanan komutları koşturur.

Bölüm Özeti

- Bu bölümde, temel bir bilgisayar yapısı belli önkabullerle tanıtılmış, işlemlerinin hızlı bellek transfer işlemleri ile nasıl gösterildiği anlatılmıştır. Bilgisayarın tasarımı çok detaya girilmeden anlatılıp, bugün kullanılan bilgisayarlarla arasındaki bağıntılar kurulmuştur. Ayrıca bilgisayarın iç yapısı ve hızlı bellekler ile gerçekleştirilen mikroişlemler arasında ilişki kurulup işletim detayları gözden geçirilmiştir.

BÖLÜM - 6

TEMEL BİLGİSAYARDA ASSEMBLER PROGRAMLAMA

Bu bölümde aşağıdaki konular incelenecektir:

- Başlangıç
- Bilgisayar yapısı ve komutlarının tanıtımı

- Makina dili, onaltılı kod, assembler dili ve yüksek seviyeli programlama dili komutlarının tanıtımı
- Assembler dili yazılım detaylarının tanıtılması
- Assembler özellikleri
- Çevrimler, aritmetik ve lojik operasyonlarla programlama örnekleri
- Kaydırma işlemleri
- Alt rutin (subroutine)
- Giriş-çıkış programlaması

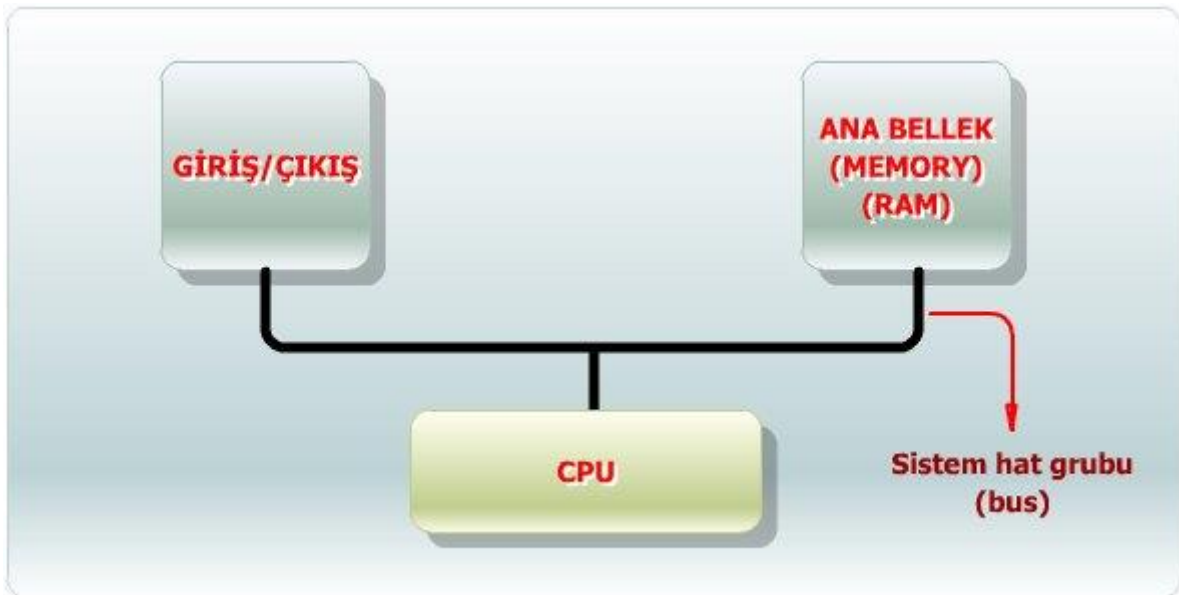
Bölüm Hedefi

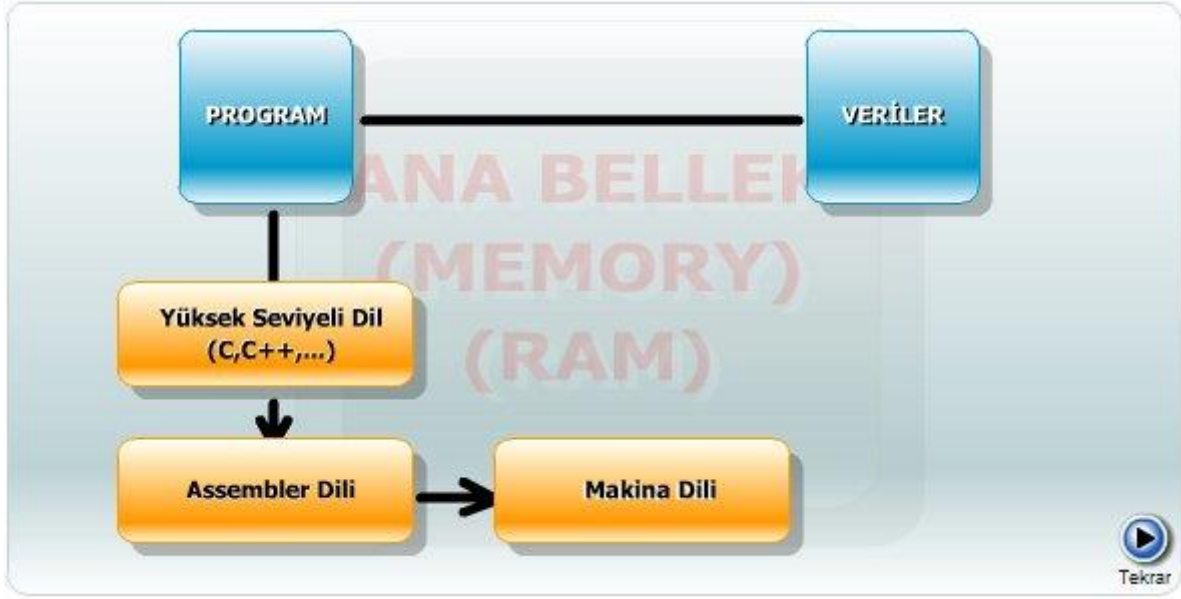
Önümüzdeki iki haftada, temel bir bilgisayarda assembler programlamanın nasıl yapıldığını örneklerle anlatacağız. Bilgisayarın tasarımından bildiğimiz komutların yapılarıyla, makina dili olarak tanıtılan ikili sayıların assembler dili haline getirilmesini açıklayacağız. Sonra çeşitli programlama örnekleri ile bu dildeki programlamanın detaylarını inceleyeceğiz.

Ayrıca, temel bilgisayarda tanımlanan tam (complete) fakat hızlı olmayan komut setinden oluşan assembler dilinin uygulamalarını göstereceğiz. Aritmetik ve lojik operasyonları, çevrim (loop) kavramını tanıtip alt rutinlerden (subroutine) bahsedeceğiz. Son olarak, giriş ve çıkış programlama örneğini tanıtacağız.

6.1 BAŞLANGIÇ

Bir sayısal bilgisayarın, donanım ve yazılım olmak üzere iki vazgeçilmez parça olduğunu belirtmiştik. Kullanılan tüm sayısal bileşenler (MUX, dekode, hızlı bellek, ana bellek vs.) donanım kısmını ve her türlü yazılan ve saklanan programlar ise yazılım kısmını oluşturmaktadır. Aşağıda temel bilgisayarın donanım ve yazılım ilişkisi açıklanmıştır. Kullanıcı tarafından yazılan ve ana bellekte saklanan program ve veri bilgisayarın işlem yapmasını sağlamaktadır. "Sistem programı" denilen ve bilgisayarın iç çalışmasını sağlayan programlar dışında, kullanıcı tarafından yazılan programlar çeşitli seviyelerde olmaktadır. "Yüksek seviyeli" (high level) programlar C, Fortran, Pascal İngilizce dilinde yazılmış, dolayısıyla, kullanıcıya daha yardımcı olan programlardır. Bugün bu programlar daha da geliştirilip, "nesne programlama" dilleri (C++, Java, Pearl, vb.) haline getirilerek İnternet'te modüler olarak kolayca koşturulabilen bir yapıya getirilmiştir.





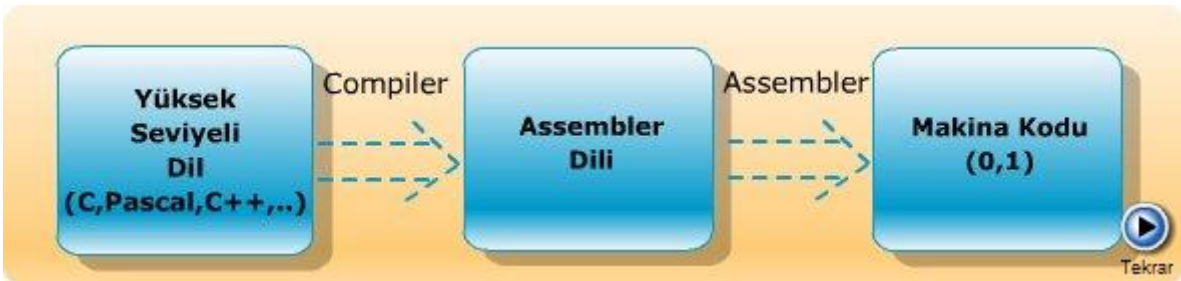
Temel bilgisayarın donanım ve yazılımı

6.1.1 Bilgisayarda Koşturulabilen Makina Kodu Elde Edilmesi

"C" gibi başka bir dil, İngilizce kolayca hatırlanabilen komutlardan oluşmaktadır. Assembler dili ise belli işlemleri yapmaya yarayan "sembol" lerden oluşmaktadır. Daha alt seviyeli bir dil grubudur. Bilgisayarın işlemcisine bağlı olarak, yapımcı firma tarafından tasarım sırasında belirlenmiştir. Makina dilindeki 0 ve 1'lerden oluşan dizileri hatırlamak veya onaltılı kodlar üzerinde çalışmak yerine, assembler dili ile çalışmak çok daha kolay olmaktadır.

Yüksek seviyeli bir dilde yazılan programı koşturabilmek için "koşturulabilen" (executable) programın veya ikili kodun (makine kodu) elde edilmesi gerekir. Bunun için derleyiciler (compiler) mevcuttur. Örneğin, C derleyicisi ile C programlama dilinde yazılmış olan bir programı koşturulabilecek formata, yani ikili makina diline döndürmek mümkün olur.

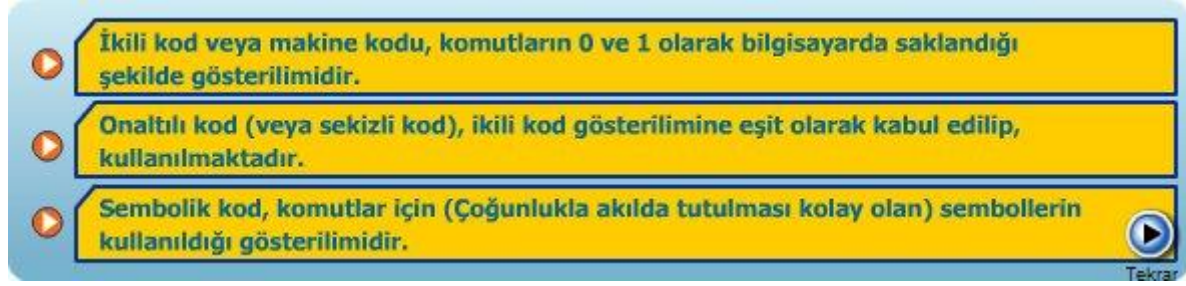
Assembler dilinde yazılmış bir program ise "assembler" vasıtasıyla makina diline çevrilmektedir. Elde edilen ikili kodlar, bilgisayar işlemcisinde koşturularak sonuçlar elde edilir ve gösterilir. Aşağıda assembler yoluyla makina kodu elde edilmesi gösterilmiştir.



Bilgisayarda koşturulabilen makina kodu elde edilmesi

6.2 MAKİNA DİLİ, ONALTILI KOD, ASSEMBLER DİLİ ve YÜKSEK SEVİYELİ PROGRAMLAMA DİLİ KOMUTLARININ TANITILMASI

Bilgisayarda yazılan programlar aşağıdaki kategorilerden birinde olabilir:



Sembolik komutlar özel bir program yoluyla ikili koda çevrilir. Bu programa "assembler" denir. Bu sembollerin oluşturduğu programa ise "assembler dili" adı verilir.

Yüksek seviyeli programlama dili, kullanıcının bir problemi çözümünde düşünme yoluna yakın olarak kullanacağı ifadelerden oluşur. Örneğin, C dilinde "for", "while" ve "switch" anahtar kelimeleri düşünme mantığına uygun sayılabilecek kelimelerdir.

6.2.1 Temel Bilgisayarın Assembler Komutları

Aşağıdaki tabloda temel bilgisayarda kullanılan tüm assembler komutları gösterilmiştir. Bu komutlar akılda tutulması çok daha kolay sembollerdir.

KOMUT ADI	AÇIKLAMA
AND	Bellek kelimesi ile AC lojik "and"
ADD	Bellek kelimesi ile AC' yi "topla"
LDA	Bellek kelimesini AC'ye yükle
STA	AC'ye bellek kelimesini yükle
BUN	Koşulsuz dallanma
BSA	Dallan ve geri dönüş adresini sakla
ISZ	Arttır ve sıfır ise sonraki komuta atla
CLA	AC'yi temizle
CLE	E bayrağını temizle
CMA	komplement AC
CME	komplement E
CIR	AC'yi sağa dairesel kaydır

CIL	AC'yi sola dairesel kaydır
INC	AC'yi arttır
SPA	Eğer AC>0 ise sonraki komuta atla
SNA	Eğer AC<0 ise sonraki komuta atla
SZE	Eğer E=0 ise sonraki komuta atla
HLT	Halt
INP	AC'ye karakter gir (INPR'den)
OUT	AC'den karakter gönder (OUTR'ye)
SKI	Giriş_bayrağı=1 ise sonraki komuta atla
SKO	Çıkış_bayrağı=1 ise sonraki komuta atla
ION	Kesinti (interrupt) geçerli (on)
IOF	Kesinti geçersiz (off)

6.2.2 İkili Kod ve Assembler Kodu Arasındaki İlişki

İkili kod, onaltılı kod ve assembler kod arasındaki ilişkiyi göstermek için örnek olarak iki komutun üç koddaki değerleri aşağıda gösterilmiştir. Bilgisayar gösterilimi yerine "STA 012" ve "LDA 002" komutlarını kullanmak çok daha uygundur.

KOMUT	İKİLİ KOD	ONALTIK KOD
STA 012	(0011 0000 0000 0110) ₂	(3006) _H
LDA 002	(0010 0000 0000 0010) ₂	(2002) _H

6.2.3 Karşılaştırmalı Program Örneği

Son olarak C dilinde yazılmış, iki sayının toplamını alan bir programın temel bilgisayar assembler dilinde yazılımını göstererek aralarındaki farkları belirtelim: C dilinde iki sayının toplamı aşağıda gösterilmiştir.

```
main ()
{
    int x=-20, y=40; /* Toplanacak iki sayı ve değerleri */

    int z; /* Sonuç değişkeni */

    z = x + y ; /* Toplama işlemi */
}
```



```
}
```

Aynı sayıları tanımladığımız assembler dilinde toplarsak, aşağıdaki programı yazmamız gerekir.

İki dilde yazılan küçük programlar karşılaştırılınca C dilinde yazılan programın ne kadar kısa olduğu gözlenebilir. Bu programın komutları tamamen anlaşılabilir ifadelerdir.

Assembler komutları daha temel (alt seviye) komutlar olup, her bir operasyonu yapabilmek için hızlı bellek transferlerini tanımlamak gerekir.

ORG 0	/ Programın başlangıcı 0 belirtilmiştir
LDA x	/ x' i AC e yükle
ADD y	/ y' yi topla
STA z	/ Sonucu z'ye sakla
HLT	/ bilgisayarı durdur
x, DEC -20	/ Onlu operand x
y, DEC 40	/ Onlu operand y
z, DEC 0	/ Onlu operand z
END	/Assembler programının sonu

Temel bilgisayar assembleri ile iki sayının toplanması

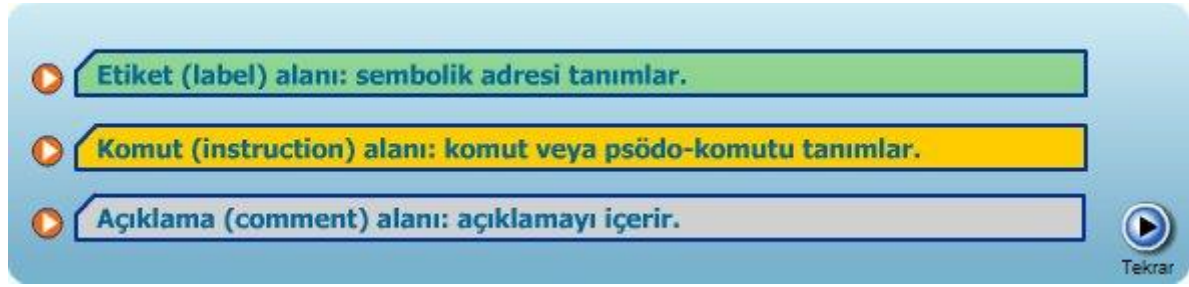
Yukarıdaki assembler programı -20 ile 40 sayısını toplamak için bölüm 5'te tanıtılmış olan AC (akümülatör) hızlı belleğini kullanmaktadır. Burada x=-20 değeri önce AC'ye saklanır, sonra ana bellekteki y=40 değeri ile ADD assembler komutu aracılığı ile toplanır. Sonuçta elde edilen sayı Z adresinde saklanır.

x,y,z etiketleri ile tanımlanan on'lu (DEC) operand değerlerinin assembler programda ayrı bir yerde gösterildiğine dikkat edilmelidir.

6.3 ASSEMBLER DİLİ YAZILIM DETAYLARININ TANITILMASI

Tüm programlama dilleri gibi, burada tanıtmış olduğumuz assembler dili de belli kurallardan oluşur. Yazılan programların doğru çevrilmesi (assembling) için, kullanıcının bu kurallara tamamiyle uyması gerekir.

Burada tanıtılan assemblerın temel birimi "satır kodu" dur.



Temel bilgisayar assemblerı ile iki sayının toplanması örneğinde END komutundan önceki üç satırda, başlangıç ifadeleri x, y, z etiketler ve etiket alanını, END, LDA, vb. komut alanını ve / (ters kesme) ile başlayan kısım ise açıklama kısmını gösterir.

6.3.1 Assembler Diline Ait Örnek

Aşağıda bir örnek satır kodu ve alanları gösterilmiştir.

Etiket	Komut	Açıklama
	STA z	/ Sonucu z'ye sakla
y	DEC 40	/ Onlu operand y

Çeşitli örneklerde tanımlanan semboller de komut alanında kullanılır:

CMA	/ Komplement AC , ana bellek referanslı olmayan komut
AND OPR	/ AND operand , doğrudan ana bellek referanslı adres
ADD PTR I	/ ADD dolaylı referanslı operand , dolaylı referanslı adres

Burada kullanılan **OPR**: operand, **PTR**: gösterici (pointer) ve **I**: dolaylı (indirect) anlamında kullanılan anahtar kelimelerdir.

Aşağıdaki tabloda bazı komutlar ve açıklamaları verilmektedir:

ORG M	M onaltılı sayısı, sonraki satırda komut veya bellek adresini belirtir
END	Programın sonunu belirtir
DEC N	İkili hale dönüştürülmüş işaretli onlu sayı
HEX N	İkili hale dönüştürülmüş işaretli onaltılı sayı

6.3.1.1 Assembler Diline Ait Örnek Soru

ÖRNEK

- $z = (x - 2 * y)$ işleminin assembler programını yazınız.

ÇÖZÜM

Örneğin Çözümü

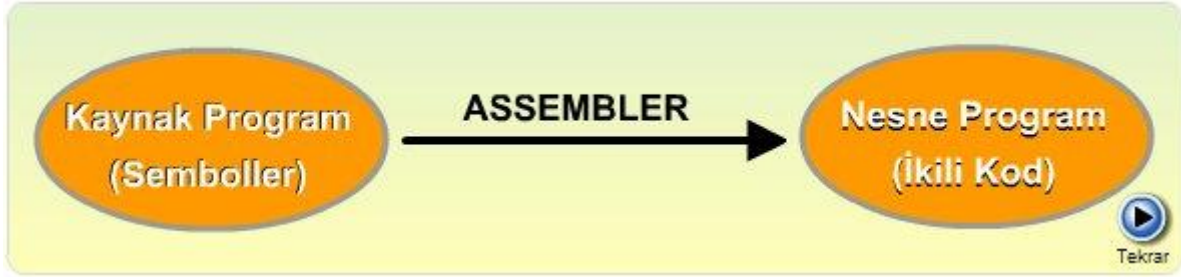
ORG 100	/ Programın başlangıcı 100 belirtilmiştir
LDA y	/ y'yi AC'ye yükle
CIL	/Dairesel sola kaydırma (E bayrağı yoluyla)
CMA	/Komplement AC
INC	/AC'yi artır
ADD x	/ x'i topla
STA z	/ Sonucu z'ye sakla
HLT	/ Bilgisayarı durdur
x, DEC 20	/ Onlu operand x
y, DEC 40	/ Onlu operand y
z, DEC 0	/ Onlu operand z
END	/Assembler programının sonu

$z = (x - 2 * y)$ işleminin assembler programı

Bu programda ORG 100 ve END psödo komutları programın sonunu başını belirtir. Değerleri 20 ve 40 olan x ve y sayıları etiketlerle HLT operasyonundan sonra tanımlanmışlardır. Yapılan işlem $(x - 2 * y)$ işlemi olup, sonuç z'ye saklanır. Burada bir 2 ile çarpma işlemi CIL ile tanımlanmıştır. Sonra ise, CMA ve INC komutları ile 2-nin komplementi alınarak, çıkarma işlemi yapılmıştır. Bu işlemin sonucu $z = (40 - 2 * 20) = 0$ olarak elde edilir.

6.4 ASSEMBLER ÖZELLİKLERİ

Assembler sembolik dili programından ikili makina kodu üreten programa verilen isimdir. Girişteki sembolik programa "kaynak program" (source program), çıkışta üretilen ikili koda da "nesne program" (object program) adı verilir. Aşağıda satır kodu (line of code) temelli assembler tanıtılmıştır.



Satır kodu temelli assembler

6.4.1 Sembollerden İkili Koda Geçiş

Sembollerden ikili koda geçiş, iki geçişli assembler (two pass assembler) programı ile yapılır. İki geçişli assembler, sembolik programı iki kere geçer veya tarar: birinci geçiş ve ikinci geçiş. Birinci geçişte, kullanıcının (veya programcının) tanımlamış olduğu semboller ile ikili karşılıkları arasında ilişki kuracak bir tablo oluşturulur. İkinci geçişte ise ikili değerlere dönüşüm yapılır.

Birinci geçişte, komutların yerlerinin takip edilmesi için yer sayıcı (location counter-LC) kullanılır. Yer sayıcı (LC) son komutun yerini gösterir. ORG psödo kodu LC'yi sıfırlar. Her bir "satır kodu" işlem gördükten sonra, sonraki satır için LC bir artırılır.

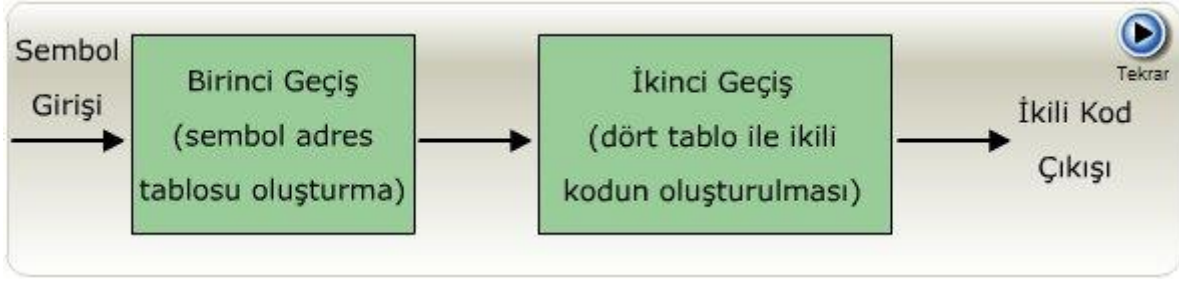
Birinci geçişte gerçekleştirilen işlev aşağıdaki gibi tanımlanabilir: LC başlangıçta 0 yapılır. Sembolik kodun etiket kısmı olup olmadığı kontrol edilir (etiketten sonra virgöl olup olmadığı araştırılır). Eğer etiket yoksa, komut alanında ORG veya END psödo komutları olup olmadığı araştırılır. Eğer ORG ise LC=0, eğer END ise birinci geçiş sonlandırılır. Eğer etiket varsa, etiket ikili eşdeğeri ve LC değeri ile birlikte "adres-sembol" tablosuna saklanır.

İkinci geçişte, bilgisayar komutları "tablo-bakma" işlemi ile ikili sayılara dönüştürülür. Tablo-bakma işlemi, tabloda saklı olan içeriklerin girdi değerleri yoluyla bulunması işlemidir. Assembler dört tablo kullanır:

- Ana bellekten bir komutun alınması (fetch cycle)
- Komutun dekode edilmesi (decode cycle)
- Efektif adresin hesaplanması ve komutun dolaylı adresinin olup olmadığının araştırılması
- Komutun koşturulması (execution cycle)

Psödo komut tablosunun girdileri dört sembolden ibarettir: ORG, END, DEC ve HEX. Her bir sembol assemblerin ilgili alt rutininin koşturulmasını sağlar. Ana bellek referanslı komutlar tablosu, temel bilgisayar tasarımından hatırlanacağı gibi, 7 eleman içerir. Ana bellek referanslı olmayan komutlar tablosu ise kalan 18 elemanı içerir. Adres-sembol tablosu ise, bilindiği gibi, birinci geçiş sırasında oluşturulur.

İkinci geçiş sırasında, karşılaşılan her bir sembol, hangi tabloya ait olduğu bütün olasılıklar kontrol edilerek araştırılır ve sonuçta ikili koda veya makina koduna dönüştürme işlemi END psödo komutuna ulaşılan kadar devam eder. Aşağıda Assemblerin işlevi blok diyagramla açıklanmaktadır.



Assemblerın işlevi

6.4.2 Assembler ile Hataların Belirlenmesi (Error Diagnostic)

Assembler'ın bir diğer önemli görevi, sembolik programda oluşabilecek hataları kontrol etmektir.

HATA	AÇIKLAMA
komut tablolarında bulunmayan bir sembolün belirlenmesi	Bu durumda, assembler sembolü ikili sayıya dönüştüremez ve bir "hata mesajı" verir.
programda etiket olarak tanımlanmamış bir sembolik adresin bulunması	Assembler bu durumda satır kodunu uygun olarak dönüştüremeyecektir, çünkü ikili eşdeğer sayı adres-sembol tablosundan bulunamayacaktır

Burada anlatılan assembler'ın basit, pratikte ise çok daha karmaşık bir program olduğuna dikkat etmek gerekir. Bugünkü bilgisayarlar kullanıcıya çok daha fazla esneklik vermektedir. Daha akılcı ve daha çok özelliği bulunan assembler'lar kullanıcıya daha kolay assembler program yazma olanağı sağlamaktadır.

Son olarak, derleyici (compiler) ile assembler arasındaki farkı belirlemek gerekir. Derleyici yüksek seviyeli bir dilden (C gibi) makina koduna dönüşüm yapar. Derleyici yapısı ve çalışması daha karmaşıktır. Derleyiciyi daha iyi anlamak için "sistem programlaması" bilmek gerekir. Bazı derleyiciler ise ara adım olarak assembler kullanmaktadır.

6.5 ÇEVİRİMLER, ARİTMETİK, LOJİK OPERASYONLAR ile PROGRAMLAMA

Bu bölümde temel bilgisayar assembler dili ile çeşitli program örneklerini göstereceğiz: Bunlardan ilki, ana bellekte arka arkaya saklanmış 20 sayıyı toplayan programdır. C dilinde 20 tane sayıyı toplamak için yazacağımız program aşağıdaki gibi olur.

```

main()
{
    int sum, a[20], i, K=20;

    sum=0;

    for ( i=0; i<=K-1; i++)

        sum= sum+a[i];

}

```

Programda "for" çevrimi 20 defa koşturularak "sum" değeri a[20] dizisinin 20 elemanının toplanmasını sağlar.

Aşağıda aynı programın assembler dilinde yazılımı görülmektedir. Program başlangıcı ORG psödo komutu ile 200 olarak tanımlanmıştır. Ana bellek adresi 250'den itibaren ise 20 operand saklanmıştır. Ana bellek adresi 400'de, bir gösterici (PTR) yer almaktadır. Bu gösterici yoluyla 20 operanda ulaşılmakta ve bir SAYICI ile endeks değerleri kontrol edilmektedir. Çevrim, aynı isimli etiketli satırla başlar ve koşulsuz dallanma komutu (BUN) ile sonlanır. Çevrim içersinde, 20 sayı sıra ile "ADD PTR I" komutu ile AC'ye toplanır, arttır ve 0 ise sonraki komuta atla (ISZ) komutlarıyla gerek sayı olarak, gerekse içeriğinin 0'dan farklı olması ile kontrol edilir. Koşullar sağlandığında çevrimden çıkılıp, AC değeri TOPLAM adresine saklanır.

ORG 200	/ Programın başlangıcı 200 belirtilmiştir
LDA ADR	/ ADR' yi AC'ye yükle
STA PTR	/ PTR'ye sakla
LDA SAYI	/ -20'yi yükle
STA SAYICI	/ SAYICI'ya yükle
CLA	/ AC'yi temizle
ÇEVİRİM, ADD PTR I	/ AC yoluyla operandları topla
ISZ PTR	/ PTR'yi arttır
ISZ CTR	/ CTR'yi arttır
BUN ÇEVİRİM	/Çevrimi tekrarla

STA TOPLAM	/ Toplam TOPLAM'a saklanır
HLT	/ Bilgisayarı durdur
ADR, HEX 250	/ 20 operandın ilk adresi
PTR, HEX 400	/ Göstericinin (PTR) yeri
SAYI, DEC -20	/ SAYICI'nın başlangıç değeri
SAYICI, HEX 0	/ SAYICI'nın yeri
ORG 250	/Operandlar 250'den başlayarak saklanmıştır
DEC 18	/ İlk operand
.....	
.....	
DEC 63	/ Son (20' nci) operand
END	/Assembler programının sonu

Assembler programında tanımlanan ADR, PTR, SAYI ve SAYICI etiketleri çeşitli HEX ve DEC değerlerin tanımlanmasında kullanılmaktadır.

6.5.1 Assembler Dilinde Çarpma ve Bölme İşlemi

Daha önce yazılan programda 2 ile çarpma işleminin, temel bilgisayar komutları kullanılarak, sola kaydırma olarak gerçekleştirildiğini belirttik. 2'ye bölme işlemi için ise sağa kaydırma kullanılır. Kullandığımız assembler dilinde birden fazla komut ile gerçekleştirilen çarpma ve bölme işlemi, bugün kullanılan kişisel bilgisayarlarda çarpma ve bölme işlemi yapan devreler nedeniyle tek bir komutla (örneğin MUL, DIV gibi) hızlı olarak gerçekleştirilebilmektedir. Ayrıca günümüz bilgisayarlarında çift haneli doğruluklu (double precision) işlemler için donanım yapıları ve komutları tasarlanmıştır. Temel bilgisayar, çift haneli doğruluklu sayıları tek haneli 2 sayı olarak yorumlayıp işlem yapar.

İkinci programlama örneğinde $(x.y)' + z$ lojik işlemini yapan programı yazalım. Aşağıda $(x.y)' + z$ lojik işlemlerini gerçekleyen program gösterilmiştir.

ORG 100	/ Programın başlangıcı 100 belirtilmiştir
LDA x	/ x'i AC'ye yükle
AND y	/ Lojik AND x ve y
STA T	/ (x.y)'yi T'ye sakla
LDA z	/ z'yi AC'ye yükle
CMA	/ Komplement AC
AND T	/ (x.y).z' ifadesini oluştur
CMA	/ (x.y).z' = (x.y)' + z sonucunu oluşturmak
STA SONUÇ	/ Sonucu SONUÇ'a sakla
HLT	/ Bilgisayarı durdur
x, BIN 1	/ ikili operand x
y, BIN 1	/ ikili operand y
z, BIN 0	/ ikili operand z
T, BIN	/ Geçici bellek elemanı
SONUÇ, BIN	/ ikili operand SONUÇ
END	/ Program sonu

Bu programda, komut setinde OR komutunun olmaması nedeniyle ilkin AND işlemi sonrada NOT (komplement) işleminin kullanıldığı izlenmelidir. Diğer bir nokta ise, ikili sayılar üzerinde tanımlanan bu işlemlerin BIN psödo kodu ile tanımlanan bellek elemanlarına uygulanmasıdır. AND lojik işleminin akümülatör AC'ye yüklenen eleman ile ana bellekte saklanan elemana uygulanmakta olduğu gözlenmelidir (ana bellek referanslı komut).

6.6 KAYDIRMA İŞLEMLERİ

Temel bilgisayarın kaydırma işlemleri CIR: dairesel sağa kaydırma ve CIL: dairesel sola kaydırmadır. Lojik kaydırma ve aritmetik kaydırma için birkaç komut kullanmak gerekir.

Lojik kaydırma, kaydırılan taraftan 0 ekleyerek yapılır. Bu ise E=0 (bayrak veya FF sıfırlanır) yapılarak sağlanır.

Lojik sağa kaydırmada kullanılan komutlar:

- CLE
- CIR

Lojik sola kaydırmada kullanılan komutlar:

- CLE
- CIL

6.6.1 Aritmetik Kaydırma

Aritmetik kaydırma, negatif sayıların gösterilimine bağlıdır. Temel bilgisayarda 2'li komplement gösterilimi kullanılır. Bu durumda, en soldaki işaret bitinin aynı kalması gerekir. Aritmetik sağ kaydırmada, E FF'sine işaret biti ile aynı değer verilir.

Aritmetik Sağa Kaydırma

- | | |
|-----|-------------------------------------|
| CLE | / E=0 |
| SPA | / Eğer AC>0 (pozitif ise) atla, E=0 |
| CME | / AC>0 (negatif) ve E=1 |
| CIR | / E ve AC'yı dairesel olarak kaydır |

Aritmetik sola kaydırmada, en az değerli bit (least significant bit) pozisyonundan 0 eklenmelidir. Bu E=0 yapılarak yapılabilir. İşaret biti kaydırma sırasında değişmemelidir. İşlem sonrası E biti ve işaret biti karşılaştırılmalıdır. Eğer iki değer aynı ise işlem doğru olarak tamamlanmıştır. Eğer farklı ise taşma (overflow) olduğunu gösterir. Kaydırma yapılmadan önceki sayı değeri çok büyük olduğundan, kaydırma sonucu akümülatör AC kapasitesi yetersiz kalır.

6.7 ALT RUTİN (SUBROUTINE)

Alt rutin (subroutine), aynı program kodunun defalarca değişik yerlerde tekrarlanmasını önler. Ortak komutlar bir kere yazılır, sonra programın değişik yerlerinden birçok defa çağrılabilir. Alt rutin bir görev veya fonksiyonun yapılması için oluşmuş komutlardan oluşur. Dallanma yoluyla ana programın herhangi bir kısmından bu alt rutine ulaşılabilir. Dönüş adresi bilgisayara saklanır ve alt rutin çalıştırdıktan sonra ana programa dönlür. Bilgisayarlar özel komutlarla alt rutine geçiş ve geri dönüşü sağlarlar.

Temel bilgisayarda BSA (dallan ve dönüş adresi sakla) komutu ana program ve alt rutin arasında geçişi sağlar. İki örnek ile alt rutinlerin nasıl çalıştığını gösterelim: birinci örnekte AC içeriğinin sola bir kaydırılmasını yapan programı yazalım. Bu işlem, temel bilgisayarda çarpma için sıklıkla kullanılacağından defalarca kullanılabilecek bir işlemdir.

Aşağıda alt rutin kullanımı gösterilmiştir. Örnek programda sola kaydırma işlemi iki kez yapılmaktadır: birincisi x, ikincisi ise y ile. Dönüş adresi ana bellekte 0 yerine saklanmaktadır. Eğer birden fazla sola kaydırma işlemi yapılması gerekirse KAYDIR alt rutini o kadar sayıda çağrılabilir. Son olarak da MSK verisi ile kaydırılan değerin en az anlamlı biti 0 yapılır.

ORG 100	/ Programın başlangıcı 100 belirtilmiştir
LDA x	/ x'i AC'ye yükle
BSA KAYDIR	/ KAYDIR alt rutinine dallan
STA x	/ x'i sakla
LDA z	/ z' yi AC'ye yükle
BSA KAYDIR	/ KAYDIR alt rutinine dallan
STA z	/ z'i sakla
HLT	/ Bilgisayarı durdur
x, HEX ABCD	/ Onaltılı operand x
y, HEX DBDB	/ İkili operand z
KAYDIR, HEX 0	/ Dönüş adresini buraya saklar
CIL	/ Dairesel sola 1 kez kaydır
AND MSK	/ AC (16) yı 0 yap
BUN KAYDIR	/ Ana programa dönüş
MSK, HEX FFFE	/ Mask operandı: en sol biti 0 yapar
END	/ Program sonu

Örnek programda, KAYDIR alt rutinin çağrılmasıyla (subroutine call) 0 adresine dönüş adresi saklanır. Alt rutin çalıştırdıktan sonra, alt rutinden dönüş (subroutine return) bu adresin geri alınması ile sağlanır.

6.7.1 Verinin Bir Adresten Diğer Adrese Taşınması

Bir alt rutin çağrıldığında, ana programın gerekli veriyi alt rutine taşıması gerekir. Örnekte bu AC yoluyla sağlanır. Alt rutin çağrılmadan önce veri AC'ye yüklenir. İşlem sonrası veri yine AC'dedir. AC bir giriş ve bir çıkış verisi için kullanılabilir ve genel olarak, alt rutin çağırılan programdan birden fazla parametreyi de alabilir. Bu da ana bellek yoluyla olabilir. Ana bellekte birbiri ardı sıra gelen yerler saklanan parametreler yoluyla parametre geçişi sağlanabilir. İkinci bir örnekle birden çok parametrenin geçişi sağlar. Bu örnekte, ardı sıra 5 elemanlık veriyi 50 adresinden 75 adresine taşıyan anaprogam tarafından çağrılan TAŞI alt rutini görelim. Aşağıda 5 elemanı bir adresten diğer adrese taşıyan programı inceleyelim.

	BSA TAŞI	/ Alt rutine dallan
	HEX 50	/ 5 elemanlık verinin ilk adresi
	HEX 75	/ 5 elemanın taşınacağı ilk adres
	DEC -5	/ Taşınacak eleman sayısı
	HLT	
TAŞI,	HEX 0	/ Alt rutin TAŞI
	LDA TAŞI I	/ Kaynak adresi AC'ye yükle
	STA PT1	/ İlk göstericiyi sakla
	ISZ TAŞI	/ Dönüş adresini arttır
	LDA TAŞI I	/ Taşınacak adresi AC'ye yükle
	STA PT2	/ İkinci göstericiyi sakla
	ISZ TAŞI	/ Dönüş adresini arttır
	LDA TAŞI I	/ Kaynak adresi AC'ye yükle
	STA Sayıcı	/ Sayıcıyı sakla
	ISZ TAŞI	/ Dönüş adresini arttır
ÇEVİRİM,	LDA PT1 I	/ Kaynaktan elemanı AC'ye yükle
	STA PT2 I	/ Taşınacak yere sakla
	ISZ PT1	/ İlk göstericiyi arttır
	ISZ PT2	/ İkinci göstericiyi arttır
	ISZ Sayıcı	/ Sayıcıyı arttır
	BUN ÇEVİRİM	/ 5 defa tekrarla
	BUN TAŞI	/ Ana programa dönüş
PT1,	-	
PT2,	-	
Sayıcı,	-	
	END	/ Program sonu

İki gösterici (pointer) yardımıyla veri bir taraftan diğer tarafa transfer edilir. Sayıcı, tanımlanan 5 elemanın bu transferini sağlar. Alt rutin işlevini tamamladıktan sonra veri 75 adresinden başlar, alt rutin ana programa geri döner ve HLT ile bilgisayar durur.

6.8 GİRİŞ/ÇIKIŞ PROGRAMLAMASI

Kullanıcılar çeşitli programlar yazarlar, bu programları koştururlar ve çıkışlarını alırlar. INP komutu ile, klavyede basılan tuştan elde edilen ASCII kodlanmış karakterleri AC (0-7)' ye transfer ederler. OUT komutu ise tersi işlem olan, AC (0-7) bitlerini yazıcı gibi, çıkış düzenine transfer eder. Aşağıdaki tabloda bir karakterin giriş ve çıkışı için program gösterilmiştir. Giriş ve çıkış bayrağını kontrol eden çevrim bayrak=0 ise devam eder, bayrak=1 olduğunda giriş veya çıkış komutlarının bulunduğu satıra atlanır. Karakter ya ana belleğe saklanır ya da basılır. Burada bu işlemin verimli bir işlem olmadığına dikkat edilmelidir. Bugün kişisel bilgisayarlar daha verimli yöntemlerle ara bellek (buffer) ve kesinti (interrupt) işlemi ile daha uygun stratejiler uygulamaktadır.

/ Bir karakterin girişi		
GİRİŞ,	SKI	/ Giriş bayrağını kontrol et
/ Bayrak=0, tekrar kontrol et		
BUN GİRİŞ		
INP		
OUT		
STA CHR		
HLT		
CHR,	-	
/ Bir karakterin çıkışı (basamak)		
LDA CHR		
/ Karakteri AC'ye yükle		
ÇIKIŞ,	SKO	/ Çıkış bayrağını kontrol et
/ Bayrak=0, tekrar kontrol et		
BUN ÇIKIŞ		
OUT		
HLT		
CHR,	HEX 00--	/ Bir ASCII karakter

ASCII karakterlerden oluşmuş bir kelimenin bilgisayara giriş veya çıkışı için kaydırma işleminin de yukarıdaki alt rutinlere eklenmesi gerekir. Tipik olarak, daha önce yapısı açıklanan sola kaydırma alt rutini arka arkaya çağrılarak tüm kelimenin INPR veya OUTR hızlı belleklerine kaydedilmesi sağlanır.

6.8.1 Kesinti İşlemi ile Giriş/Çıkış

Kesinti işlemi ile yapılan giriş ve çıkışın üstünlüğü, giriş ve çıkış düzeninin başlattığı bir kesinti ile verilerin transferinin sağlanmasıdır. Böylece bayrak kontrol etmek suretiyle kaydedilen uzun bekleme zamanının ortadan kaldırıldığı gözlenmelidir. Böylece, bilgisayar diğer işlevlerle meşgul olabilmektedir. Bu şekilde sadece kesinti işlemi sırasında bilgisayar işlemcisi giriş veya çıkış işlemi yapabilecektir. Kesinti işlevi çoklu programlama ortamlarında (birden fazla programın aynı bellekte saklanması durumunda) çok uygun olmaktadır.

Kesinti işlemi koşturulan programın giriş veya çıkış düzeninin bayrağı 1 yapmasına kadar devam eder. Bayrak=1 olduğunda, bilgisayar koşturduğu programı tamamlar ve kesintiyi onaylar. Bu sırada hızlı bellek içerikleri de saklanır. Dönüş adresi 0 adresinde saklanıp, 1 adresinde bulunan (önceden saklanan) giriş veya çıkış işlemi için kesinti servis rutini koşturulmaya başlanır. Bu işlem giriş/çıkış tamamlanana kadar sürer, sonra 0 adresinde saklanan dönüş adresine geri dönülerek, koşturulan programa kalınan yerden devam edilir. İşlemcinin hızlı bellek içerikleri, kesinti öncesi ve sonrası aynı olmalıdır. Aşağıda giriş/çıkış için kesinti işlemi blok diyagramı gösterilmiştir.



Bölüm Özeti

- Bu bölümde, temel bir bilgisayarda assembler programlamanın nasıl yapıldığı örneklerle anlatılmıştır.

BÖLÜM - 7

TEMEL BİLGİSAYARIN MİKROPROGRAMLAMA ile KONTROLÜ

Bu bölümde aşağıdaki konular incelenecektir:

- Mikroprogram ile kontrolde bellek yapısı
- Adres dizileri ile kontrol, kontrol rutinleri
- Mikroprogram ile kontrolde bilgisayar yapısı ve mikrokomutlar
- Mikroprogramlama
- Kontrol birimi tasarım detayları

Bölüm Hedefi

Önümüzdeki iki haftada, temel bir bilgisayarda mikroprogramlama ile kontrolü tanıtacağız. Donanım ile kontrolü 5. bölümde anlatmıştık. Lojik kapılar ve FF'lerle gerçekleştirilen donanım ile kontrol hızlı fakat esnek olmayan bir yapı sergilemektedir.